key:

```
map = new Map

harold = name: 'Harold'
map.set harold, age: 50

map.get harold
# {age: 50}
```

Be mindful that when you use an object as a key, only the actual object can be used to `get` from a `Map` instance, not just any old object that *looks* the same:

```
map.get {name: 'Harold'}
# undefined
```

Unfortunately, a `Map` can be a problem because it can eat memory. Sounds dangerous—what does it mean?

WeakMap

In addition to `Map` there's also a `WeakMap`. A `WeakMap` is a `Map` that is *not* enumerable. The reason it's not enumerable is so that objects referenced by it can be garbage collected if the only reference to the object is in the `WeakMap` instance itself:

```
stackOfPapers =
  paperOnGladiators:
    text:
      "Gladiators, it seems, were fat."

papersMap = new WeakMap
papersMap.add stackOfPapers.paperOnGladiators

delete stackOfPapers.paperOnGladiators
```

When that's implemented using a `WeakMap`, then deleting the `paperOnGladiators` from the `stackOfPapers` allows it to be garbage collected. But if it were implemented using a regular `Map`, then deleting the object wouldn't make it available for garbage collection because the `Map` has a strong reference to the deleted paper. You can see